

Overview

Part 15 —  AI Evaluation · 15-ai-evals/README.md

Module 15 — AI Evaluation

Where you are: Stop 15 of 16 on the end-to-end AI track. **Time:** ~16–20 hours · **Prereq:** Module 01 [01-ml-foundations] (metrics, leakage, honest evaluation) and Module 04 [04-llm] (how LLMs actually fail) — this module assumes both.

Every module before this one produces a system that does something. This module is how you find out whether it actually works, and keep knowing that as it changes. Classical ML evaluation (Module 01) is a solved problem with sharp edges: pick the right metric, split honestly, hunt leakage. LLM evaluation inherits every one of those lessons and adds three new, structural problems — the output is open-ended natural language instead of a fixed label, the grader you reach for is often itself a model with its own biases, and the benchmarks everyone quotes decay the moment they're famous enough to leak into training data. This module is the theory underneath all of that: how to build a custom eval suite from real failures, how to build and calibrate an LLM-as-judge you can trust, how to tell a real 2-point improvement from sampling noise, and how to wire the result into a workflow that actually improves your system instead of producing a dashboard nobody checks.

Learning objectives

By the end of this module you can:

1. Explain why LLM evaluation is structurally harder than classical ML evaluation, and name benchmark contamination as a specific, unavoidable-without-vigilance failure mode.
2. Read a benchmark leaderboard (MMLU, HumanEval, SWE-bench, GPQA, Arena) critically, knowing each one's documented weaknesses instead of quoting its number uncritically.
3. Build a custom eval suite from real production failures using the eval pyramid (deterministic -> LLM-as-judge -> human), and size a golden dataset for the delta you actually need to detect.
4. Design a binary-criteria judge rubric, name the standard bias catalog (position, verbosity, leniency, self-preference, format), and calibrate a judge against human labels using Cohen's kappa.
5. Attach a Wilson confidence interval and a paired significance test (McNemar / paired bootstrap) to any eval score before trusting a delta between two versions — and recognize p-hacking your own eval set.
6. Describe online evaluation (sampling, guardrail metrics, spend caps) and human evaluation done well (structured rubrics, inter-annotator agreement, where to spend scarce human attention).
7. Assemble all of the above into an eval-driven development workflow: CI gates, dataset versioning, and evals as a living asset with an owner, not a one-time launch report.

Suggested path

#	Do this	File	Time
1	Why LLM eval is different, benchmark literacy, the eval pyramid	notes/01-core-concepts.md	2.5h
2	Judge craft, statistical rigor, eval-driven development	notes/02-judge-craft-and-eval-driven-development.md	3h
3	Run the stats lab — Wilson CI, McNemar, sample-size trap, p-hacking	code/statistical_rigor_for_evals.py	1.5h
4	Run the judge lab — bias injection, kappa, binary vs. Likert	code/judge_bias_and_calibration.py	1.5h
5	Slides	slides/	1h
6	Exercises	lab/EXERCISES.md	5h
7	Papers	papers/PAPERS.md	4h
8	See the tooling that implements this end to end	../16-ai-tech-stack/tracks/langsmith/	2h

Where this fits with the rest of the track

- `../01-ml-foundations/` — the metrics/leakage/calibration baseline this whole module extends. Read it first if you haven't.
- `../04-llm/` — where benchmark contamination originates (pretraining data) and the failure modes (hallucination, sycophancy) this module measures.
- `../08-rag/` — RAG-specific metrics (faithfulness, context precision/recall, RAGAS). Referenced here, not repeated — that module owns retrieval-specific evaluation.
- `../06-ai-agents/` — trajectory evaluation and compounding per-step error for agentic systems. Referenced here, not repeated.
- `../13-ai-security/` — red-team and adversarial eval suites.
- `../12-ai-governance/` — audit trails and sign-off requirements for eval results in regulated contexts.
- `../16-ai-tech-stack/tracks/langsmith/` — the tooling implementation of everything taught here: datasets, `evaluate()`, CI gates, online evaluators, annotation queues. This module is the theory it's built on; read that track once you can explain the theory back.

The 14 terms you must own

benchmark contamination · LLM-as-judge · rubric · binary criteria · position bias · verbosity bias · leniency drift · Cohen's kappa · Wilson score interval · McNemar's test · paired bootstrap · p-hacking · eval pyramid · golden dataset

Exit check

You can hand a colleague: (1) a calibrated judge rubric with a written-down kappa against human labels, (2) an eval score reported with a confidence interval and, for any A/B comparison, a paired significance test —

not a bare percentage, and (3) a one-paragraph plan for how that eval set grows from real production failures and gates CI, rather than existing as a one-time report.